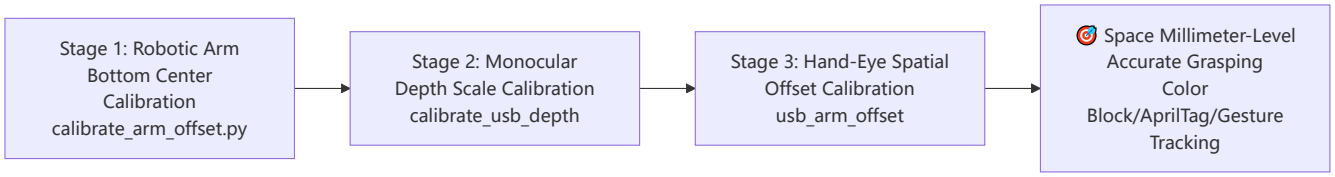
# USB Camera Calibration, Robotic Arm Center and Hand-Eye Spatial Offset Calibration

## 1. Calibration System and Process Overview

Before using **USB monocular camera** for advanced embodied tasks such as robotic arm 3D spatial grasping, color block sorting, and machine code tracking, the entire system must complete "from hardware bottom to visual upper" **three core calibrations**:




1. **Stage 1: Robotic Arm Bottom Servo Physical Center Calibration** (`calibrate_arm_offset.py`):
    - The servo metal output shaft spline has discrete tooth pitch, and after installation, there is tiny mechanical tolerance between the physical zero position and algorithm theoretical baseline.
    - **You must first calibrate the robotic arm each joint physical center** (joints 1~5 at 90° vertical alignment, **joint 6 gripper at 180° completely closed**), ensuring that the robotic arm forward/inverse kinematics (FK/IK) theoretical geometric posture strictly matches the actual physical posture.
2. **Stage 2: USB Monocular Ranging Depth Scale Calibration** (`calibrate_usb_depth`):
    - Using small hole imaging geometric model and known physical prior size, calibrate the monocular camera's depth scaling factor (`depth_scale`), achieving high-precision physical distance measurement of the monocular camera's material spatial Z-axis.
3. **Stage 3: Robotic Arm Hand-Eye Grasping Spatial Offset Calibration** (`usb_arm_offset`):
    - Camera is fixed at the end of the robotic arm (Eye-in-Hand structure). Through hand-eye homogeneous transformation matrix and robotic arm forward kinematics (FK), calibrate the fixed physical spatial offset (X,Y) between the gripper center and visual detection center in the world coordinate system, eliminating camera installation geometric tolerance.

---

## 2. Calibration Stage 1: Robotic Arm Bottom Servo Physical Center Calibration

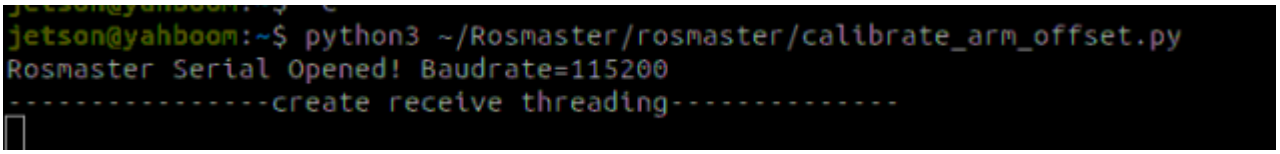
> **Important**
>
> Robotic arm hand-eye calibration relies on robotic arm forward kinematics solution (`get_end_point_mat`). If there is angle zero position deviation in the robotic arm bottom joints, the calculated camera spatial pose will have serious drift, directly causing subsequent hand-eye grasping offset! Therefore, **robotic arm center calibration is the first indispensable step for all visual grasping**.

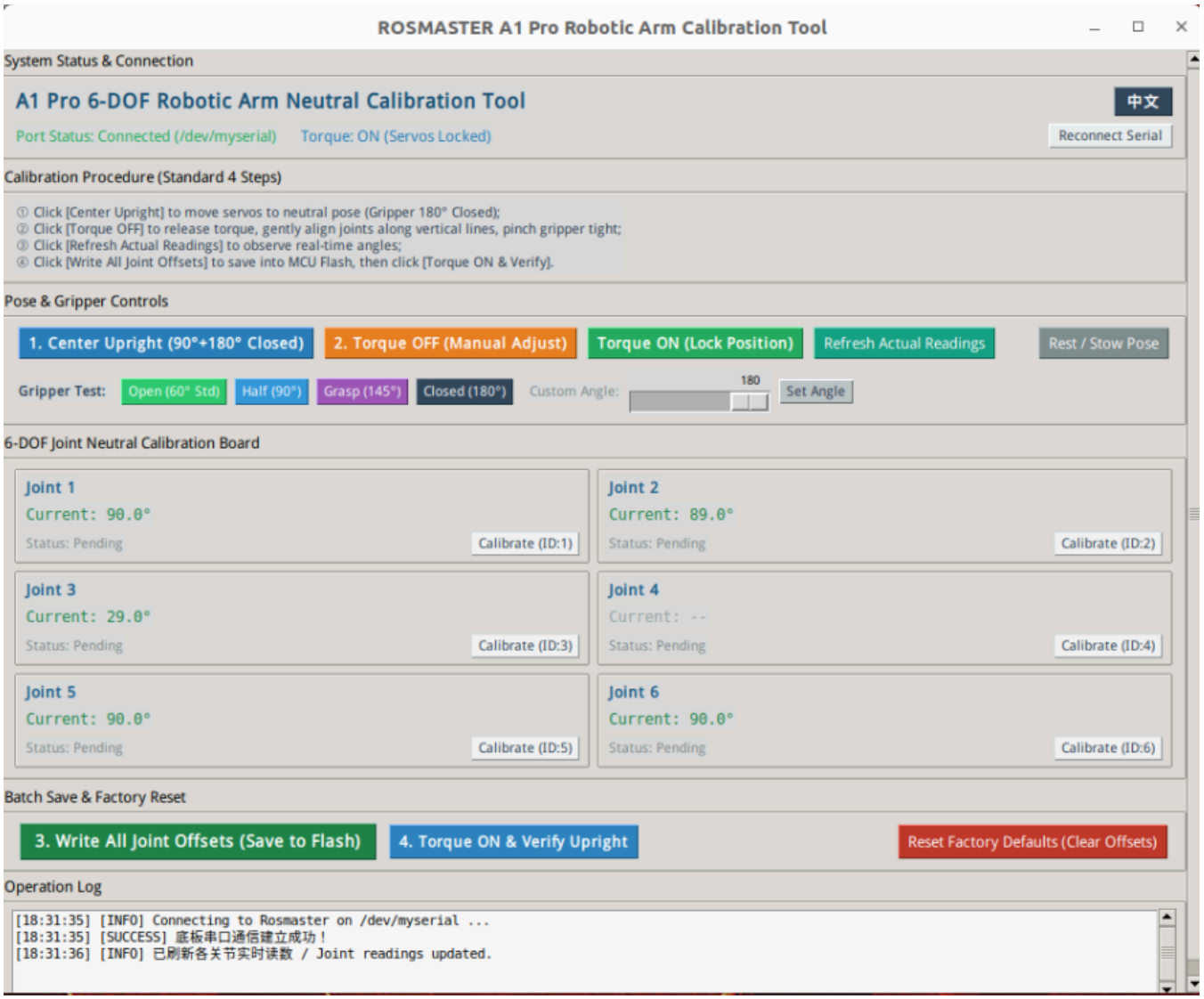
## 2.1 Start Robotic Arm Center Calibration Program

The program supports modern dark GUI graphical and pure SSH command-line adaptive interaction two modes:

```
# Method A: Desktop / VNC environment graphical startup (Recommended)
python3 ~/Rosmaster/rosmaster/calibrate_arm_offset.py

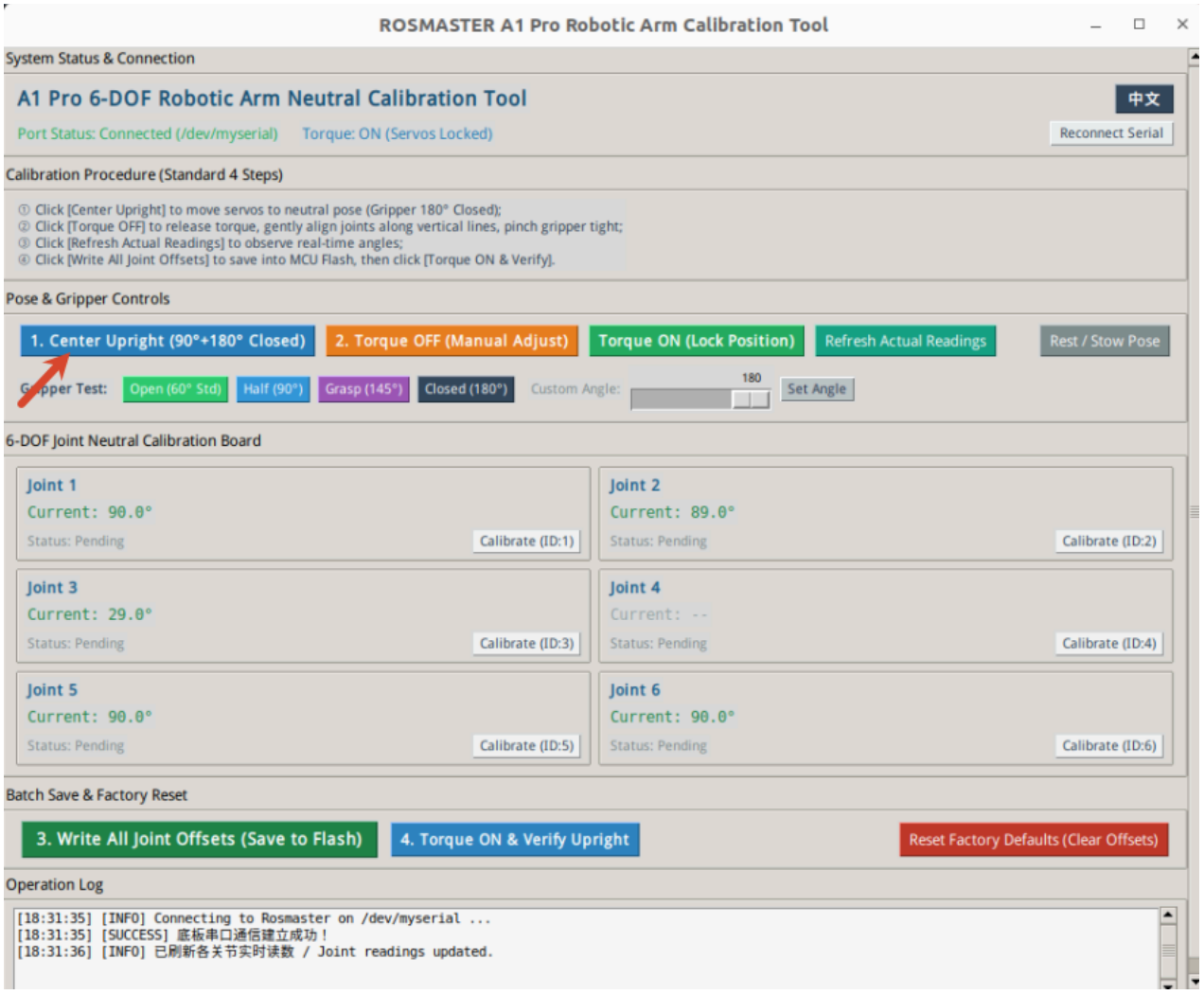
# Method B: Pure SSH terminal desktop-free startup
python3 ~/Rosmaster/rosmaster/calibrate_arm_offset.py --cli
```



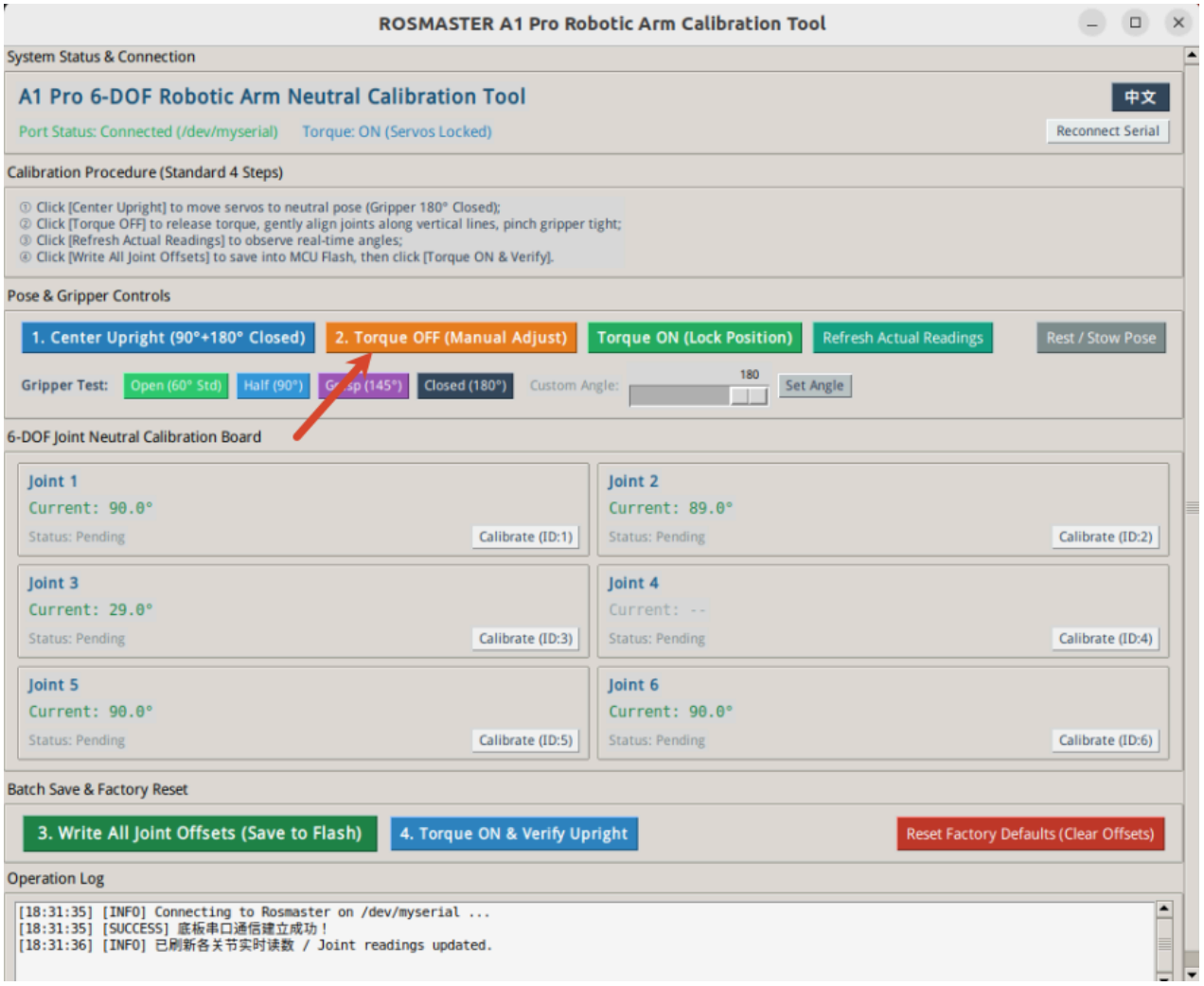




## 2.2 Core Calibration Four-Step Operation

1. **Let robotic arm return to neutral straight position**: Click **【1. Center Upright (90°+180° Closed)】** (CLI press `1`), the single-chip microcomputer sends initial theoretical angle `[90, 90, 90, 90, 90, 180]` (gripper at 180° closed position);

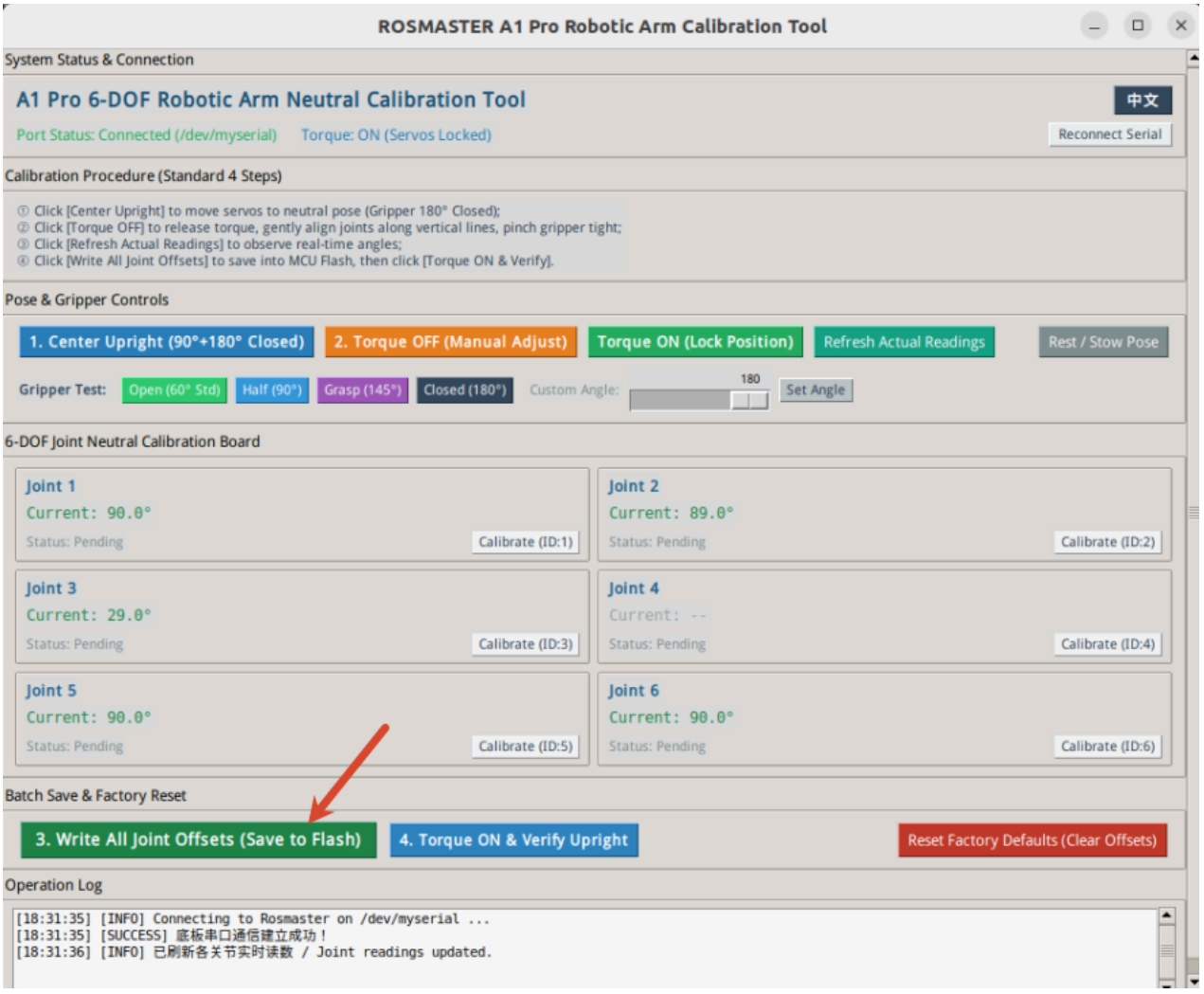



2. **Turn off servo torque**: Click **【2. Torque OFF (Manual Adjust)】** (CLI press `2`), each joint motor releases force, **gently adjust each joint by hand to strictly align with the mechanical vertical line, close joint 6 gripper to 180° aligned**;


ROSMASTER A1 Pro Robotic Arm Calibration Tool
System Status & Connection
A1 Pro 6-DOF Robotic Arm Neutral Calibration Tool
中文
Port Status: Connected (/dev/myserial)
Torque: ON (Servos Locked)
Reconnect Serial
Calibration Procedure (Standard 4 Steps)
① Click [Center Upright] to move servos to neutral pose (Gripper 180° Closed);
② Click [Torque OFF] to release torque, gently align joints along vertical lines, pinch gripper tight;
③ Click [Refresh Actual Readings] to observe real-time angles;
④ Click [Write All Joint Offsets] to save into MCU Flash, then click [Torque ON & Verify].
Pose & Gripper Controls
1. Center Upright (90°+180° Closed)
2. Torque OFF (Manual Adjust)
Torque ON (Lock Position)
Refresh Actual Readings
Rest / Stow Pose
Gripper Test:
Open (60° Std)
Half (90°)
Closed (180°)
Custom Angle:
180
Set Angle
6-DOF Joint Neutral Calibration Board
Joint 1
Current: 90.0°
Status: Pending
Calibrate (ID:1)
Joint 2
Current: 89.0°
Status: Pending
Calibrate (ID:2)
Joint 3
Current: 29.0°
Status: Pending
Calibrate (ID:3)
Joint 4
Current: --
Status: Pending
Calibrate (ID:4)
Joint 5
Current: 90.0°
Status: Pending
Calibrate (ID:5)
Joint 6
Current: 90.0°
Status: Pending
Calibrate (ID:6)
Batch Save & Factory Reset
3. Write All Joint Offsets (Save to Flash)
4. Torque ON & Verify Upright
Reset Factory Defaults (Clear Offsets)
Operation Log
[18:31:35] [INFO] Connecting to Rosmaster on /dev/myserial ...
[18:31:35] [SUCCESS] 底板串口通信建立成功！
[18:31:36] [INFO] 已刷新各关节实时读数 / Joint readings updated.



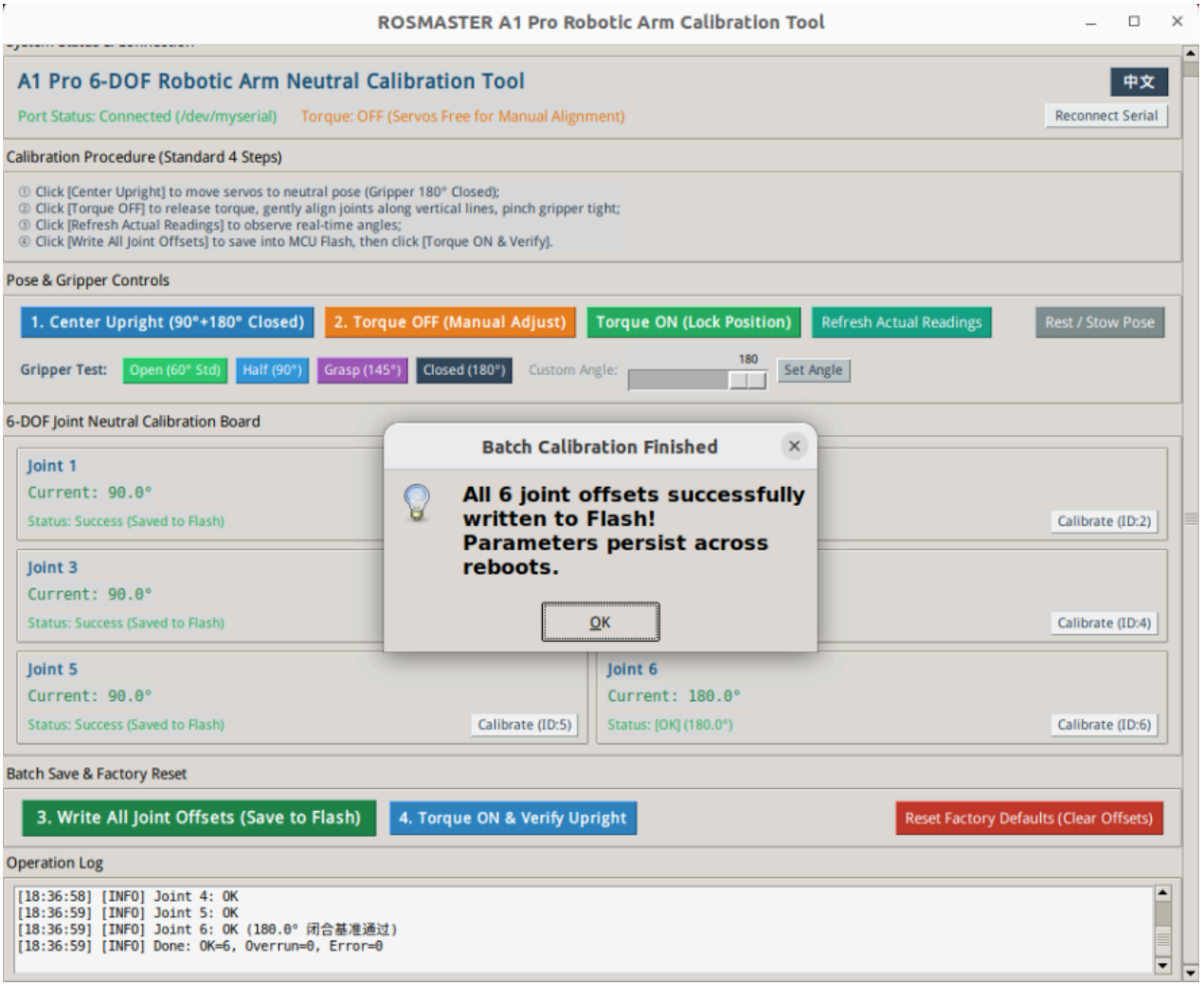
3. **Write deviation save Flash**: Click **【3. Write All Joint Offsets (Save to Flash)】** (CLI press `4`), the chassis reads the current actual encoder readings of each servo, calculates the Offset and permanently stores it in the single-chip microcomputer Flash (not lost after power off);



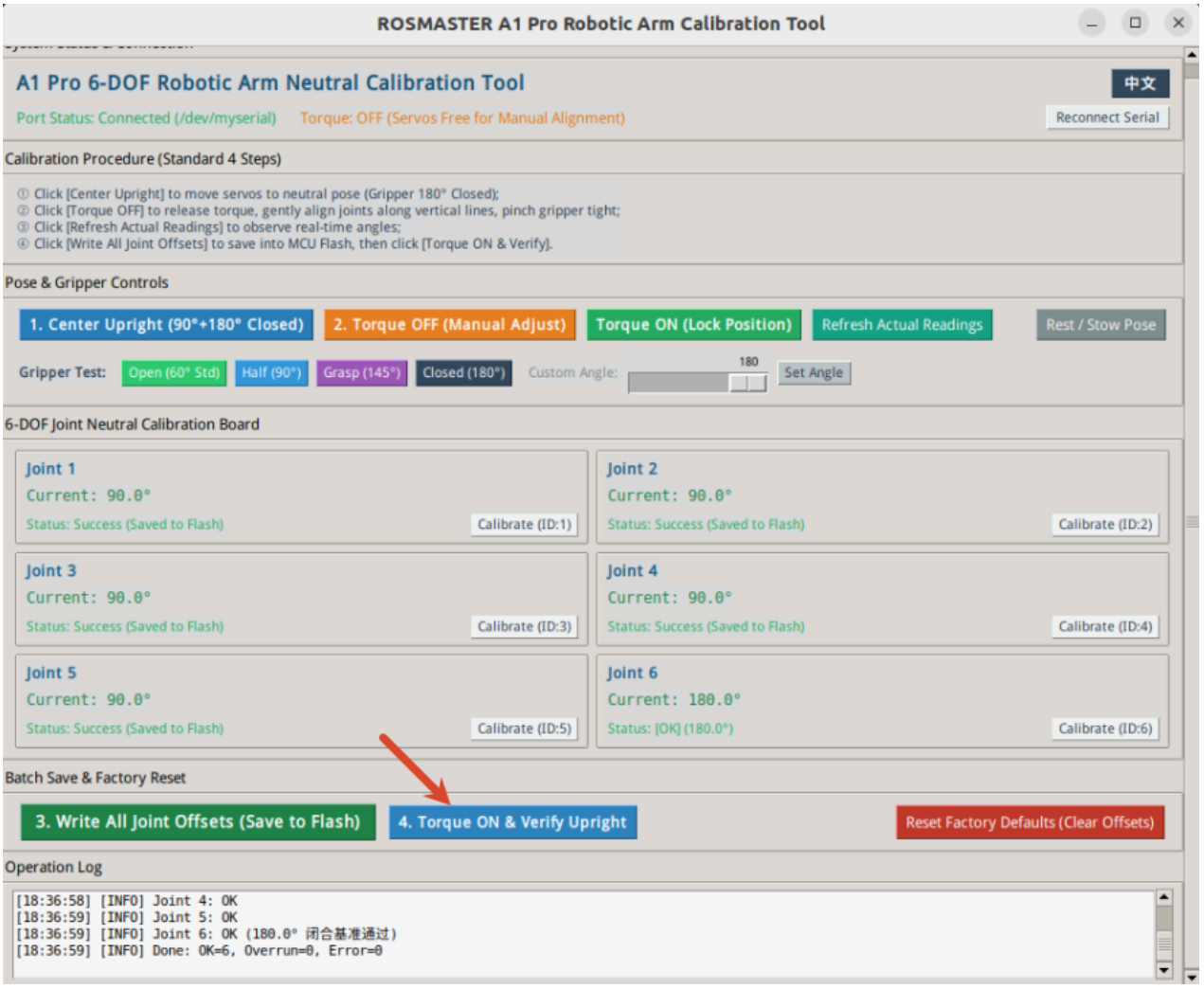

Click YES to confirm




Success will show




4. **Turn on torque verification**: Click **【4. Torque ON & Verify Upright】** (CLI press `7`), power on and lock, verify if the robotic arm is as straight as a benchmark and the gripper is completely closed.




# 3. Calibration Stage 2: USB Monocular Depth Scale Calibration

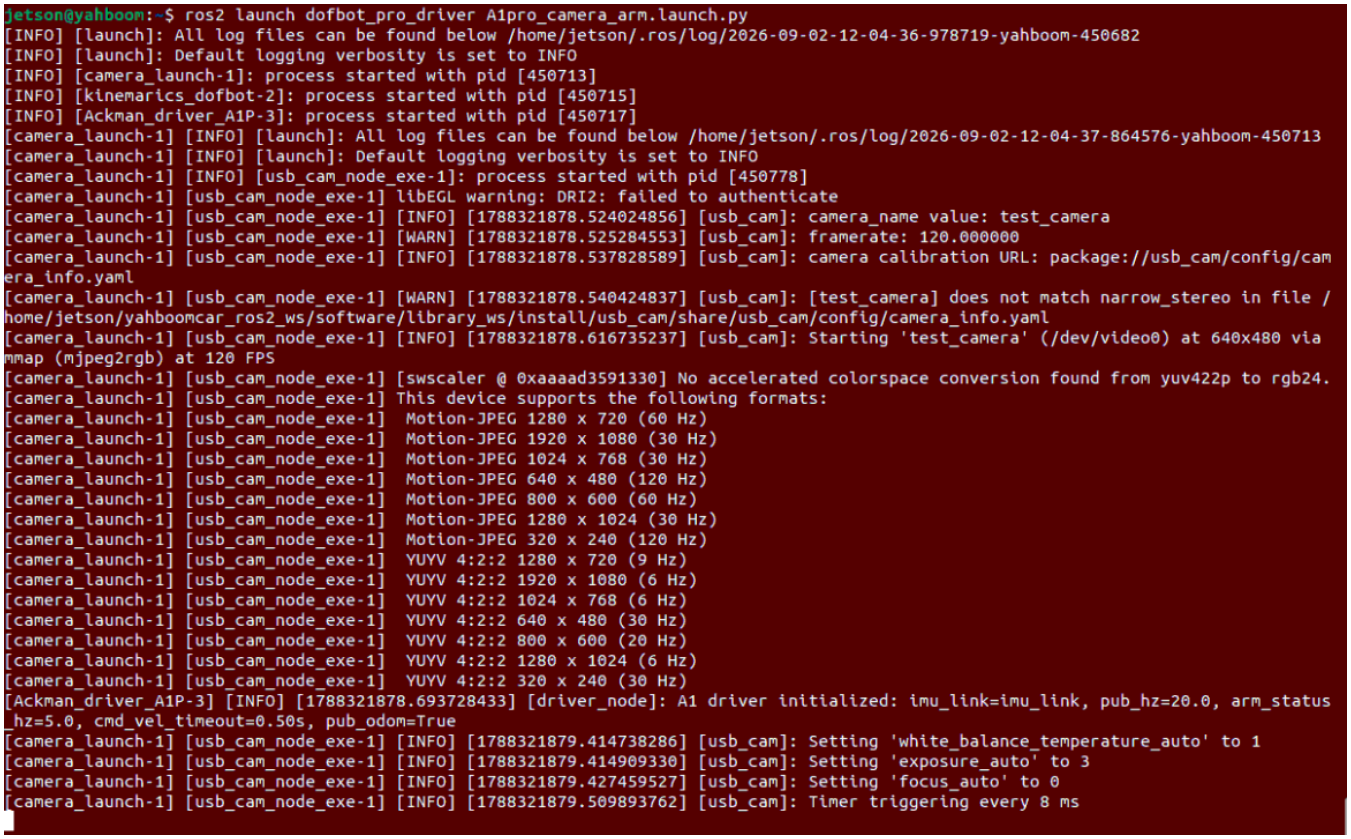
The USB monocular camera itself has no hardware distance sensor. The system calculates the original depth through the known physical size of the material (such as AprilTag side length, color block side length) and the pixel size on the image plane based on the small hole imaging geometric model, then calculates the calibrated high-precision physical depth through the depth scaling factor ( `depth_scale` ) obtained from this calibration program:

> **Monocular Ranging Calculation Formula**:
>
> ```
> Original depth Z_raw = (Camera focal length f × Material actual physical size W_real) / Image pixel size W_pixel
> Calibrated physical depth Z = Z_raw × depth_scale
> ```
>
> - **f**: Camera focal length (pixels);
> - **W_real**: Known physical size of calibration material (meters, e.g., AprilTag is 0.02m, color block is 0.03m);
> - **W_pixel**: Pixel width of the algorithm-extracted outline in the image (pixels);
> - **depth_scale**: Depth scaling factor calculated by this calibration program's fitting.

## 3.1 Preparation and Environment Startup

Start USB camera driver and robotic arm basic driver in **Terminal 1** (no need to start inverse solution service):

```
ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
```




> **Tip**
>
> Ensure the camera focal length is adjusted clearly, the material is centered in the screen without overexposure or serious reflection.

## 3.2 Start Depth Calibration Program

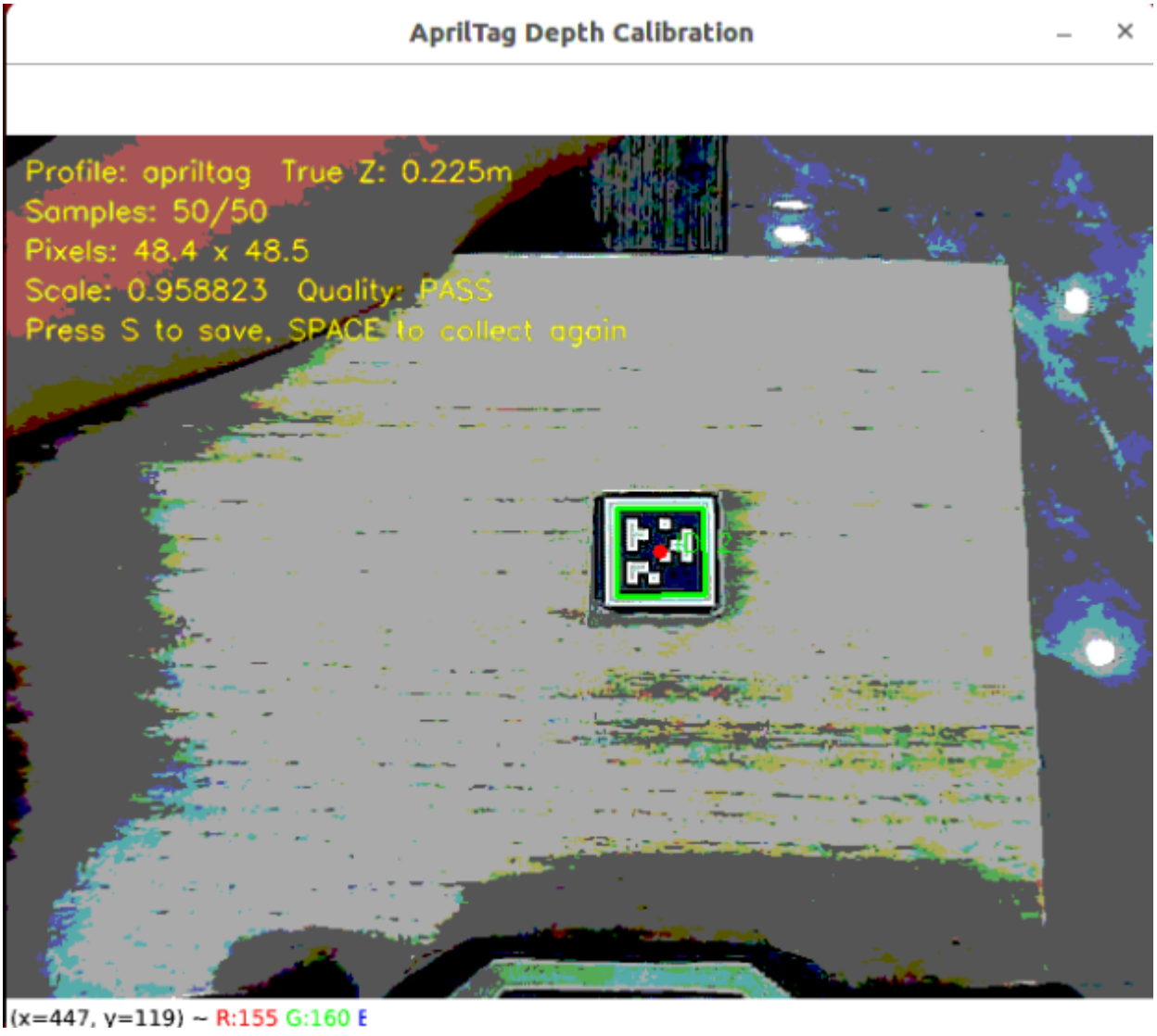
**Three modes for this calibration, corresponding to three different types of cases; each mode will output different depth information, all three modes need to be completed separately.**

### Mode A: AprilTag Machine Code Calibration (Recommended, highest precision)

Use the matching 2x2 cm machine code wooden block placed directly below the camera (that is, the matching 3×3 side length machine code wooden block):

```
# If using 2x2cm (0.02m) machine code wooden block, can pass parameter to specify size:
ros2 run dofbot_pro_usb calibrate_usb_depth --mode apriltag --ros-args -p target_width:=0.02 -p target_height:=0.02
```

```
jetson@yahboom:~$ ros2 run dofbot_pro_usb calibrate_usb_depth --mode apriltag --ros-args -p t
arget_width:=0.02 -p target_height:=0.02
[INFO] [1788321983.208558102] [apriltag_depth_calibration]: apriltag depth calibration ready:
 depth=0.2250m, samples=50, config=/home/jetson/yahboomcar_ros2_ws/dofbot_pro_ws/src/dofbot_p
ro_usb/config/usb_camera_params.yaml
[INFO] [1788321983.236771841] [apriltag_depth_calibration]: moving arm to initial joints: [90
, 100, 0, 0, 90, 90]
[INFO] [1788321985.775917398] [apriltag_depth_calibration]: arm is ready; calibration is enab
led
[INFO] [1788321989.758381802] [apriltag_depth_calibration]: sample collection started
[INFO] [1788321993.319421959] [apriltag_depth_calibration]: calibration complete: scale=0.958
823, width_scale=0.957813, height_scale=0.959407, raw_depth=0.2347m, axis_diff=0.17%, MAD=0.0
6%
```



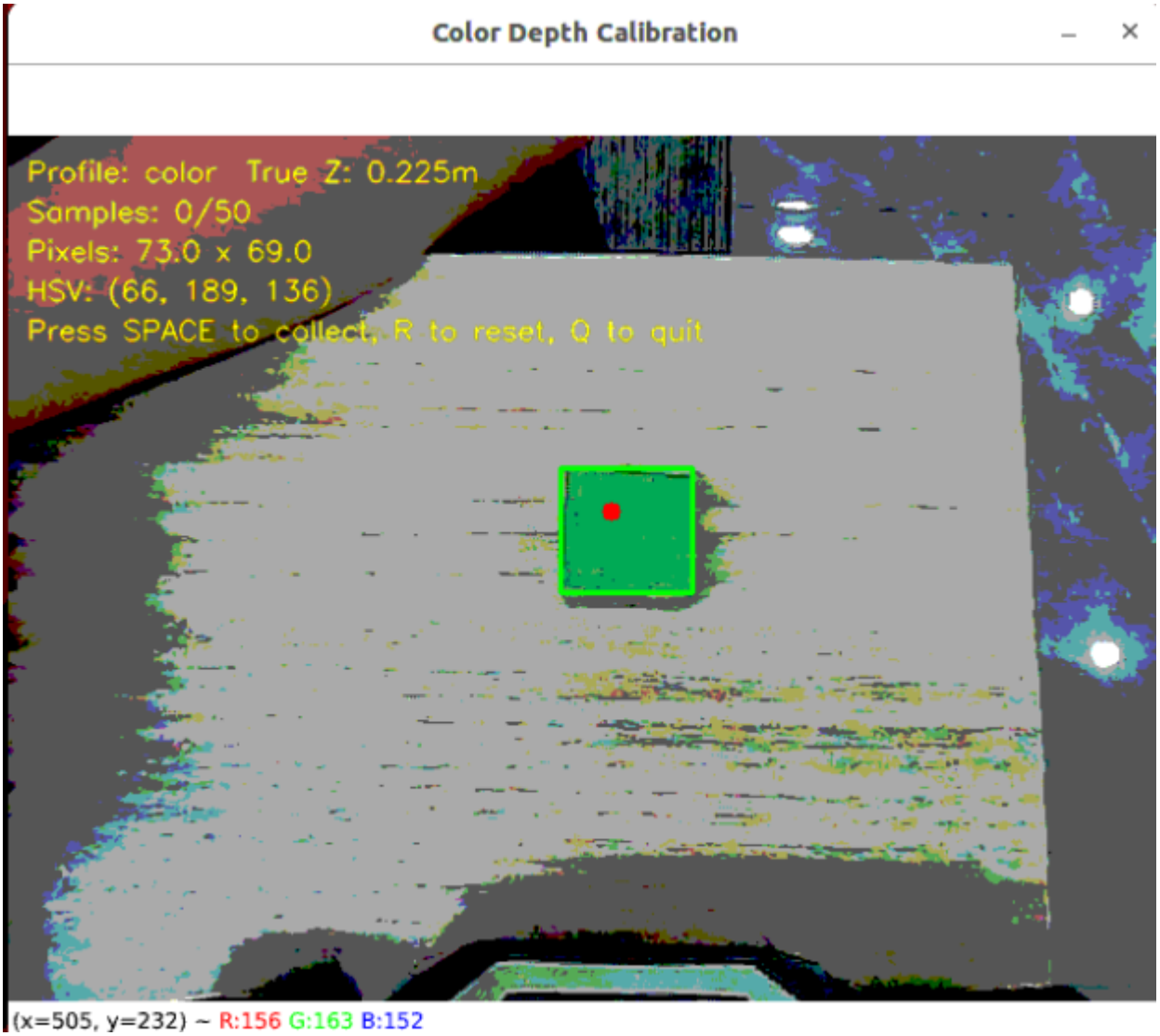

1. **Save configuration**: Press `S` **key**, calibration result automatically writes to configuration file;
2. **Exit program**: Press `Q` **key** or `Ctrl+C` in terminal to exit.

## Mode B: Color Block Color Calibration

Use the specified color cubic wooden block (default 3x3 cm):

```
ros2 run dofbot_pro_usb calibrate_usb_depth --mode color --ros-args -p target_width:=0.03 -p
target_height:=0.03
```

After startup, **left-click the target color block** in the image window, the system will automatically lock that color's HSV threshold and extract the external rectangle.




1. **Save configuration**: Press `S` **key**, calibration result automatically writes to configuration file;
2. **Exit program**: Press `Q` **key** or `Ctrl+C` in terminal to exit.

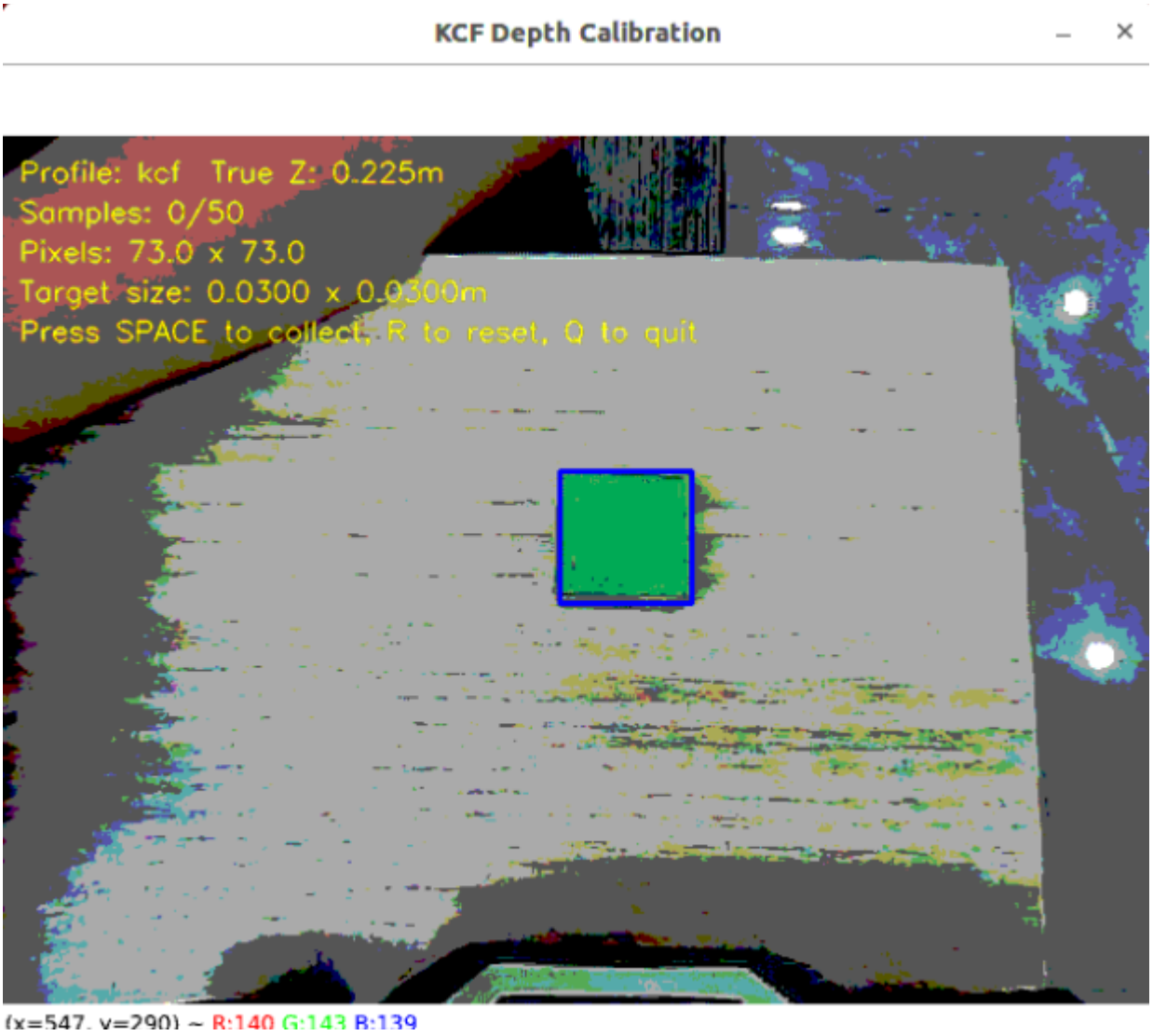
## Mode C: KCF Target Tracking Box Selection Calibration

Used for any object with clear geometric shape (default 3x3 cm):

```
ros2 run dofbot_pro_usb calibrate_usb_depth --mode kcf --ros-args -p target_width:=0.03 -p target_height:=0.03
```

```
jetson@yahboom:~$ ros2 run dofbot_pro_usb calibrate_usb_depth --mode kcf --ros-args -p target_width:=0.03 -p target_height:=0.03
[INFO] [1788322338.477530067] [kcf_depth_calibration]: kcf depth calibration ready: depth=0.250m, samples=50, config=/home/jetson/yahboomcar_ros2_ws/dofbot_pro_ws/src/dofbot_pro_usb/config/usb_camera_params.yaml
[INFO] [1788322338.513721972] [kcf_depth_calibration]: moving arm to initial joints: [90, 100, 0, 0, 90, 90]
[INFO] [1788322341.117144074] [kcf_depth_calibration]: arm is ready; calibration is enabled
```

After startup, **click and drag a rectangle** around the target object in the image window.




1. **Save configuration**: Press `S` **key**, calibration result automatically writes to configuration file;
2. **Exit program**: Press `Q` **key** or `Ctrl+C` in terminal to exit.

## 3.3 Depth Calibration Parameter Configuration File Description

Calibration result is saved at:

```
dofbot_pro_ws/src/dofbot_pro_usb/config/usb_camera_params.yaml
```

```yaml
depth_calibration:
  apriltag:
    depth_scale: 0.959314
    depth_bias: 0.0
  color:
    depth_scale: 0.936504
    depth_bias: 0.0
```

# 4. Calibration Stage 3: Robotic Arm Hand-Eye Grasping Offset Calibration

After completing robotic arm center calibration and monocular depth calibration, the camera can precisely calculate the material's spatial distance in the **camera coordinate system**. Since there is a fixed physical distance between the camera bracket installation structure and the gripper closing center, it is necessary to further calibrate the spatial geometric compensation value of the camera center and gripper center in the **robotic arm world base coordinate system**. When calibrating, please **use the matching 4×4 cm machine code wooden block**.

## 4.1 Start Driver and Kinematics Environment

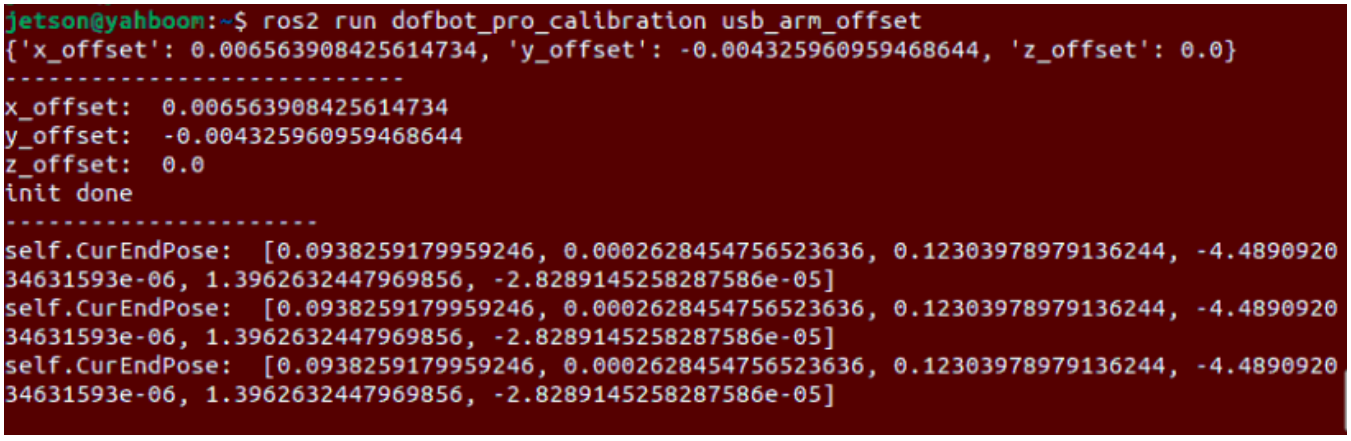
Start camera, robotic arm driver and KDL inverse kinematics solution node in **Terminal 1**:

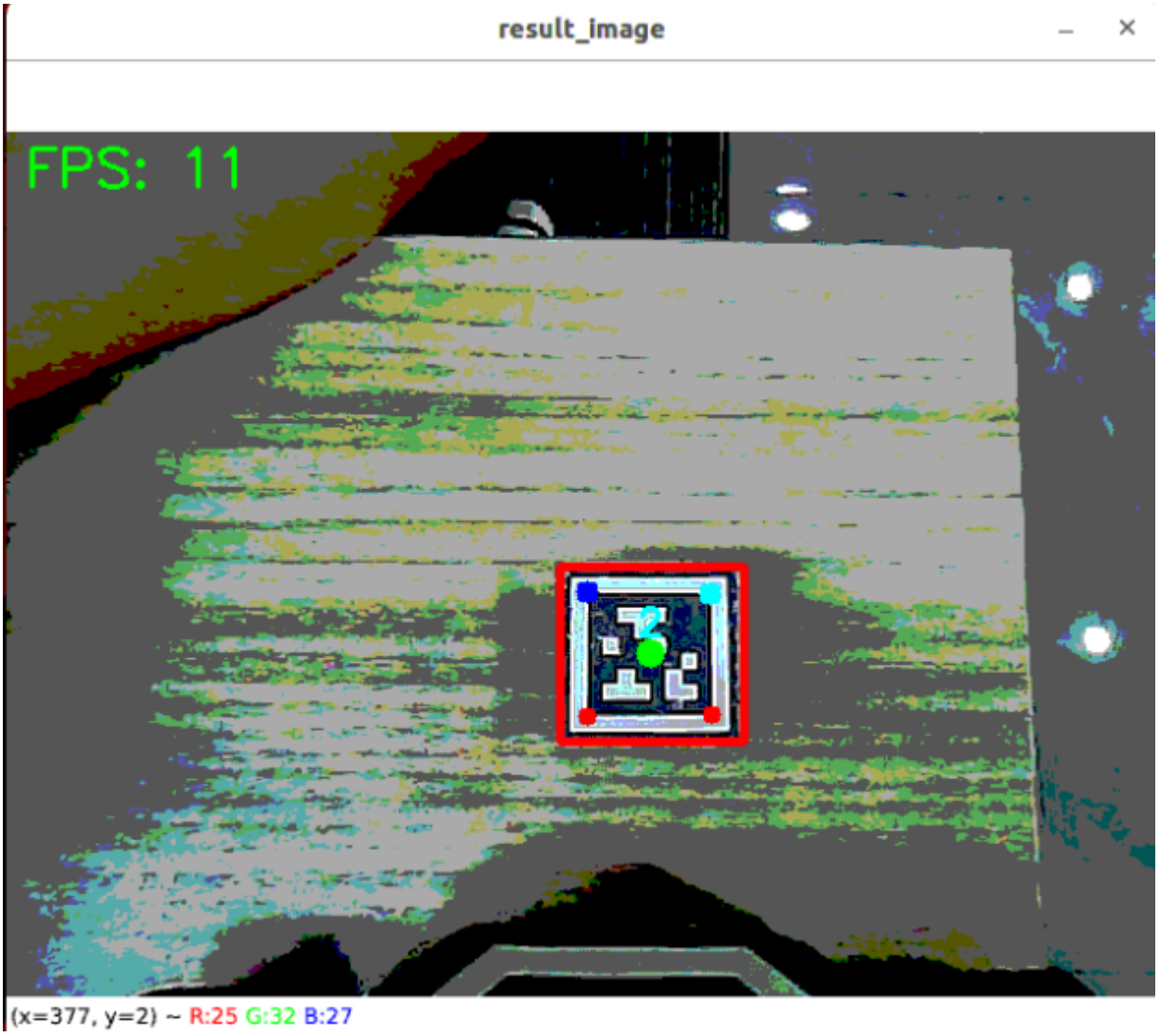
```
ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
```

## 4.2 Start Grasping Offset Calibration Node

Run hand-eye offset calibration program in **Terminal 2**:

```
ros2 run dofbot_pro_calibration usb_arm_offset
```






## 4.3 Practical Calibration Steps

1. **Observe calibration window**:
    - After program starts, the robotic arm will automatically move to the calibration inspection pose `[90°, 100, 0, 0, 90°, 90°]`;
    - A **red calibration alignment rectangle** will appear in the image (pixel area coordinates `(307, 241)` to `(409, 338)`);
2. **Align reference frame**:
    - Place the **4×4 cm AprilTag machine code wooden block** on the desktop, slightly adjust the block position so that it is exactly at the **center of the red rectangle** in the image;
3. **Confirm stable recognition**:
    - A green detection outline, center crosshair, and real-time calculated spatial distance `dist_detect` will be drawn in the image;
4. **Trigger save**:
    - Keep the image window in an active state, press ** `Space key (SPACE)**`;
    - Terminal outputs calculation completion prompt:

```
Finish calibrate offset (only X and Y). Z keeps previous value.
```

- The program will automatically write the calculated offset to the configuration file.

## 4.4 Offset Configuration File Description

Calibration result is saved at:

```
dofbot_pro_ws/src/dofbot_pro_driver/config/usb_offset_value.yaml
```

```
x_offset: 0.009368814795573138
y_offset: 0.007718452024610358
z_offset: 0.0
```

If there is a small fixed deviation when actually grasping certain items in the future, you can also directly edit this file to fine-tune `x_offset` and `y_offset` values.

---

# 5. Core Source Code Deep Analysis

## 5.1 Depth Calibration Algorithm and Robust Statistics (`calibrate_usb_depth.py`)

Source location: `dofbot_pro_ws/src/dofbot_pro_usb/dofbot_pro_usb/calibrate_usb_depth.py`

```
def calculate_depth_scale_sample(true_depth, fx, fy, width_px, height_px, model_width,
model_height):
    # Combine X/Y two axes to calculate theoretical distance
    z_width = float(fx) * float(model_width) / float(width_px)
    z_height = float(fy) * float(model_height) / float(height_px)
    raw_depth = 0.5 * (z_width + z_height)

    # Calibration scaling factor = true distance / calculated raw distance
    scale = float(true_depth) / raw_depth
    return DepthScaleSample(width_px, height_px, raw_depth, scale, ...)
```

## 5.2 Hand-Eye Calibration and Spatial Pose Solution (`usb_arm_offset.py`)

Source location:
`dofbot_pro_ws/src/dofbot_pro_calibration/dofbot_pro_calibration/usb_arm_offset.py`

### 1. Camera to Robotic Arm End Homogeneous Transformation Matrix (`EndToCamMat`)

Camera is fixed at the end of the robotic arm (Eye-in-Hand), installation pose matrix defined as:

```
self.EndToCamMat = np.array([
    [ 0,  0,  1, -0.101],
    [-1,  0, -0.028,  0.002],
    [ 0, -1,  0,  0.048],
    [ 0,  0,  0,  1.0  ]
])
```

## 2. World Coordinate System Material 3D Position Solution

```
def compute_heigh_from_location(self, camera_location):
    # 1. Camera coordinate system -> Robotic arm end coordinate system
    PoseEndMat = np.matmul(self.EndToCamMat, self.xyz_euler_to_mat(camera_location, (0, 0,
0)))
    # 2. Robotic arm forward kinematics (FK) to get end pose matrix in base world coordinate
system
    EndPointMat = self.get_end_point_mat()
    # 3. Convert to robotic arm base world coordinate system
    WorldPose = np.matmul(EndPointMat, PoseEndMat)
    pose_T, pose_R = self.mat_to_xyz_euler(WorldPose)
    return pose_T  # [X_world, Y_world, Z_world]
```

## 3. Offset Update and Storage

The detected world coordinate and theoretical preset reference point ($XX = 0.145m, Y = -0.013m$) are differentiated, the difference is the geometric compensation value for all subsequent spatial grasping tasks:

```
x_offset_value = 0.145 - pose[0]
y_offset_value = -0.013 - pose[1]

offset_config['x_offset'] = float(x_offset_value)
offset_config['y_offset'] = float(y_offset_value)
with open(offset_file, 'w') as file:
    yaml.dump(offset_config, file, default_flow_style=False)
```